



DATABASE PROGRAMMING (SECP3623)

SEM I – 2025/2025

Exploring and Comparing Types of Indexes

Learning Outcomes

Students will be able to:

- Explain the concept, advantages, and disadvantages of various index types.
- Create and apply indexes in MySQL using SQL statements.
- Demonstrate how indexes improve query performance.
- Compare different index types using real-world examples.

Group Formation

- Form a **group of three (3)** members.
- Each group must submit **one PowerPoint (PPT) file**.
- The group name and members' details must appear on the cover page.

Instructions

1. Create a new database in MySQL using the following commands:
CREATE DATABASE index_lab;
USE index_lab;
2. Complete all questions for each selected index topic. For every related SQL question, you must include:
 - The SQL command(s) used.
 - A screenshot of the output after executing the SQL command.
 - A short explanation (2–3 sentences) describing the result and explaining what it demonstrates.
3. Each group must choose **ONE topic only** from the following list:

	TOPIC	DESCRIPTION
1.	UNIQUE INDEXES	How unique indexes enforce data integrity and avoid duplicate entries.
2.	MULTICOLUMN (COMPOSITE) INDEXES	How composite indexes work and how column order affects performance.
3.	B-TREE INDEXES	How B-Tree indexes store data hierarchically for faster searching and sorting.
4.	BITMAP INDEXES	How bitmap indexing improves query performance for low-cardinality data.
5.	TEXT (FULLTEXT) INDEXES	How text indexes improve full-text search efficiency.

Note: Refer to the detailed questions provided in the Appendices (A-E). Include SQL code, screenshots, and reflections for each question.

APPENDIX A: UNIQUE INDEXES

1. Define **Unique Index** and explain how it differs from **Primary Key**.
2. List three advantages and three disadvantages of Unique Indexes.
3. Create a table customer (cust_id INT AUTO_INCREMENT, email VARCHAR(50), phone VARCHAR(15), PRIMARY KEY(cust_id));.
4. Insert 10 records (including a duplicate email) and observe the result.
5. Add a unique index on email:

```
ALTER TABLE customer ADD UNIQUE idx_email (email);
```
6. Try inserting a duplicate email again and record the error.
7. Use SHOW INDEX FROM customer; and explain the output.
8. Describe how a **unique index** improves lookup performance.
9. Compare query performance (with and without index):

```
SELECT * FROM customer WHERE email='user1@gmail.com';
```
10. Reflect: When is a **unique index** not appropriate for a column?

APPENDIX B: MULTICOLUMN (COMPOSITE) INDEXES

1. Define **multicolumn index** and explain the **Leftmost Prefix Rule**.
2. State three advantages and three disadvantages of multicolumn indexes.
3. Create table orders (order_id INT, customer_id INT, product_id INT, order_date DATE, total DECIMAL(10,2));.
4. Insert 15 sample rows with various customer_id and product_id combinations.
5. Create a multicolumn index:

```
CREATE INDEX idx_cust_prod ON orders(customer_id, product_id);
```
6. Run queries:
(a) WHERE customer_id=2 AND product_id=10;
(b) WHERE product_id=10;
Compare execution times.
7. Use EXPLAIN to show which query used the index.
8. Drop and recreate the index in reverse order. Observe impact.
9. Discuss: Why does column order matter in a multicolumn index?

10. Reflect: When should you use multiple single-column indexes instead of one multicolumn index?

APPENDIX C: B-TREE INDEXES

1. Describe what a **B-Tree index** is and how it organizes data internally.
2. List three advantages and two disadvantages of B-Tree indexes.
3. Create a table employee (emp_id INT, emp_name VARCHAR(50), department VARCHAR(30), salary DECIMAL(10,2), join_date DATE);.
4. Insert 15–20 rows of employee data.
5. Add a B-Tree index on salary:

```
CREATE INDEX idx_salary ON employee(salary);
```
6. Execute and record query time:

```
SELECT * FROM employee WHERE salary BETWEEN 3000 AND 7000;
```
7. Add an index on join_date and compare range query performance.
8. Use EXPLAIN to confirm index usage.
9. Drop the index and re-run queries. Record time difference.
10. Discuss: Why is the B-Tree index ideal for range and sorting queries?

APPENDIX D: BITMAP INDEXES

1. Explain what a **Bitmap Index** is and how it stores values internally.
2. List three advantages and three disadvantages of Bitmap Indexes.
3. Create table student (stud_id INT, name VARCHAR(30), gender CHAR(1), status VARCHAR(10));.
4. Insert 20 rows with combinations of gender (M/F) and status (Active/Inactive).
5. Create indexes on gender and status.
6. Run query:

```
SELECT COUNT(*) FROM student WHERE gender='F' AND status='Active';
```
7. Use EXPLAIN and describe how the query is processed.
8. Explain why bitmap indexes are best for low-cardinality columns.
9. Discuss what happens to bitmap indexes during frequent updates/deletes.
10. Reflect: In what type of database environment would you prefer a bitmap index (e.g., OLTP vs OLAP)?

APPENDIX E: TEXT (FULLTEXT) INDEXES

1. Define **Full-Text Index** and describe how it differs from a normal B-Tree index.
2. List three advantages and two disadvantages of Full-Text Indexes.
3. Create table articles (article_id INT, title VARCHAR(100), content TEXT);.
4. Insert 10 short articles with varying topics.
5. Create a Full-Text index on content:

```
CREATE FULLTEXT INDEX idx_content ON articles(content);
```
6. Run text search:

```
SELECT * FROM articles WHERE MATCH(content) AGAINST('machine learning');
```
7. Compare with:

```
SELECT * FROM articles WHERE content LIKE '%machine learning%';
```
8. Use EXPLAIN to verify index usage.
9. Test Boolean search mode:

```
SELECT * FROM articles WHERE MATCH(content) AGAINST('+AI -robot' IN  
BOOLEAN MODE);
```
10. Reflect: When should you use a text index instead of pattern matching with LIKE?

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1: Index Creation & Implementation	Creation of the database and table(s) for the chosen index topic using correct syntax and logical structure. Tables include valid datatypes, PK/FK where needed.	3	
	Successful creation of the selected index (e.g., UNIQUE, MULTICOLUMN, B-TREE, BITMAP, or FULLTEXT). SQL commands (CREATE INDEX, ALTER TABLE, etc.) executed and documented with screenshots.	3	
	Demonstration of valid insertions and queries showing how the index affects duplicate entries, sorting, or searching.	2	
Task 2: Performance Testing & Analysis	Queries executed with and without the index to show performance difference (e.g., using EXPLAIN, timing results).	4	
	Clear explanation of query plan and interpretation of how the index improves search or filtering efficiency.	2	
Task 3: Discussion & Reflection	Discussion of advantages and disadvantages of the selected index type.	2	
	Reflection on when this index type is or isn't appropriate in real-world database scenarios (OLTP/OLAP context).	2	
Task 4: Documentation & Presentation	PowerPoint well-organized, labelled screenshots, and readable SQL/code formatting.	2	